up at this line and just return `null`, as the result of an unsuccessful search.

But, of course, by now readers must have learnt we shouldn't despair, because this is the time we provide a better alternative!

And indeed, as we mentioned at the beginning of the chapter, we can create a collection of feature vectors for the images in our dataset, extract the feature vector for `Z`, let's call it `FZ`, and perform a search in the feature vectors space instead of searching directly the image dataset.

Now, comparing `FZ` to tens or hundreds of thousands of other vectors could also be slow and expensive, in terms of time, memory, and disk accesses.

If each memory page, stored on disk, can hold `M` feature vectors, we would have to perform `n/M` disk accesses and read `n*k` float values from disk.

And that's exactly where an SS-tree comes into play: by using an SS-tree with at most `M` entries per node, and at least `m≤M/2`, we can reduce the number of pages loaded from disk to[7] `2*logM(n)`, and the number of float values read to `~k*M*logM(n)`.

Listing 10.3 shows the pseudocode for SS-tree's `search` method; we can follow the steps on figures 10.10 and 10.11: initially node will be the root of our example tree, and so not a leaf; we'll then go directly to line #5 and start cycling through `node`'s children, in this case `S1` and `S2`.